

10. Bridge Design Pattern in Java

10.1 Introduction

The **Bridge Design Pattern** is a **structural pattern** that decouples an abstraction from its implementation, so the two can vary independently.

10.2 Problem it Solves

You have a hierarchy where one dimension (e.g., abstraction) and another (e.g., implementation) grow independently. Without Bridge, this leads to an explosion of subclasses.

10.3 What is Bridge Pattern?

Bridge pattern splits a class into two parts: **Abstraction** and **Implementation**, which are developed independently and combined at runtime.

10.4 Real-World Analogy

A **remote control** works with various **TV brands**. The remote is an abstraction, and each TV is an implementation.

10.5 Structure

```
package com.mannemlokeskesh;  
  
//Abstraction  
abstract class RemoteControl {  
    protected Device device;  
  
    public RemoteControl(Device device) {  
        this.device = device;  
    }  
  
    public abstract void togglePower();  
}  
  
//Implementation Interface  
interface Device {  
    void turnOn();  
  
    void turnOff();  
  
    boolean isEnabled();  
}
```

10.6 Benefits

- Reduces coupling between abstraction and implementation
- Enables runtime binding
- Makes code easier to extend and maintain

10.7 Implementation Steps

1. Create the **implementation interface**
2. Create concrete implementations
3. Create the **abstraction base class**
4. Extend abstraction and link with implementation

10.8 Examples

Example1: Remote Control and Device

```
package com.mannemlokes.example1;

interface Device {
    void turnOn();

    void turnOff();

    boolean isEnabled();
}

class TV implements Device {
    private boolean on = false;

    public void turnOn() {
        on = true;
        System.out.println("TV is turned ON");
    }

    public void turnOff() {
        on = false;
        System.out.println("TV is turned OFF");
    }

    public boolean isEnabled() {
        return on;
    }
}

abstract class RemoteControl {
    protected Device device;

    public RemoteControl(Device device) {
        this.device = device;
    }

    public abstract void togglePower();
}

class BasicRemote extends RemoteControl {
    public BasicRemote(Device device) {
        super(device);
    }

    public void togglePower() {
        if (device.isEnabled()) {
            device.turnOff();
        }
    }
}
```

```

        } else {
            device.turnOn();
        }
    }
}

public class Example1BridgeDemo {
    public static void main(String[] args) {
        Device tv = new TV();
        RemoteControl remote = new BasicRemote(tv);

        remote.togglePower();
        remote.togglePower();
    }
}

```

Output:

```

TV is turned ON
TV is turned OFF

```

Example2: Shape and Color Bridge

```

package com.mannemlokes.example2;

interface Color {
    String fill();
}

class Red implements Color {
    public String fill() {
        return "Color is Red";
    }
}

class Blue implements Color {
    public String fill() {
        return "Color is Blue";
    }
}

abstract class Shape {
    protected Color color;

    public Shape(Color color) {
        this.color = color;
    }

    public abstract void draw();
}

class Circle extends Shape {
    public Circle(Color color) {
        super(color);
    }

    public void draw() {
        System.out.println("Drawing Circle. " + color.fill());
    }
}

```

```

    }
}

public class Example2ShapeApp {
    public static void main(String[] args) {
        Shape redCircle = new Circle(new Red());
        Shape blueCircle = new Circle(new Blue());

        redCircle.draw();
        blueCircle.draw();
    }
}

```

Output:

```

Drawing Circle. Color is Red
Drawing Circle. Color is Blue

```

Example3: Report Formatters

```

package com.mannemlokes.example3;

interface Formatter {
    String format(String content);
}

class HTMLFormatter implements Formatter {
    public String format(String content) {
        return "<html><body>" + content + "</body></html>";
    }
}

class JSONFormatter implements Formatter {
    public String format(String content) {
        return "{ \"content\": \"" + content + "\" }";
    }
}

abstract class Report {
    protected Formatter formatter;

    public Report(Formatter formatter) {
        this.formatter = formatter;
    }

    public abstract void display(String content);
}

class FinancialReport extends Report {
    public FinancialReport(Formatter formatter) {
        super(formatter);
    }

    public void display(String content) {
        System.out.println("Financial Report: " +
                           formatter.format(content));
    }
}

```

```

public class Example3ReportApp {
    public static void main(String[] args) {
        Report htmlReport = new FinancialReport(new HTMLFormatter());
        Report jsonReport = new FinancialReport(new JSONFormatter());

        htmlReport.display("Bridge Pattern Overview");
        jsonReport.display("Bridge Pattern Overview");
    }
}

```

Output:

```

Financial Report: <html><body>Bridge Pattern Overview</body></html>
Financial Report: { "content": "Bridge Pattern Overview" }

```

10.9 Bridge vs Other Patterns

Pattern	Purpose
Bridge	Decouple abstraction from implementation
Adapter	Convert one interface to another
Strategy	Change implementation dynamically
Decorator	Add responsibilities without changing class

10.10 Pros and Cons

✓ Pros

- Abstraction and implementation can evolve independently
- Promotes loose coupling
- Easy to extend in future

✗ Cons

- Adds complexity due to more classes
- Overkill for small hierarchies

10.11 When to Use / Not to Use

Use When:

- You have abstraction and implementation that need to vary independently
- You want to avoid subclass explosion

Avoid When:

- Your class hierarchy is simple and unlikely to grow

10.12 Summary

Feature	Description
---------	-------------

Intent	Separate abstraction from implementation
Structure	Has abstraction, refined abstraction, implementor, concrete implementor
Key Benefit	Reduce coupling and improve flexibility

The **Bridge Pattern** is excellent for large-scale systems with multiple dimensions of variability.